

Peripheral Interrupts

Peripheral Interrupts

- 1. External Interrupt Overview
- 2. Core Concepts
 - 2.1 Interrupt Trigger Types
 - 2.2 IRAM_ATTR — Why ISR Must Be in RAM
 - 2.3 volatile — Compiler Optimization Barrier
 - 2.4 ISR Design Principles
- 3. Project Architecture and Flow
- 4. Hardware Dependencies
- 5. Source Code Analysis
 - 5.1 Macro Definitions and Global Variables
 - 5.2 exti_isr_cb() — Interrupt Callback (Core)
 - 5.3 led_init() — LED Initialization
 - 5.4 exti_init() — Interrupt Initialization (Three Steps)
 - 5.5 app_main() — Main Entry
- 6. Operation Flow
 - 6.1 Build and Flash
 - 6.2 Normal Operation
 - 6.3 Verification Testing
- 7. Common Issues

1. External Interrupt Overview

External Interrupt (EXTI) is a hardware mechanism where MCUs respond to external asynchronous events. When a GPIO pin detects a specified edge (rising/falling/any edge), hardware automatically pauses current program execution, jumps to a preset **Interrupt Service Routine (ISR)** to handle the event, and returns to continue original program execution after handling.

This experiment uses GPIO0 falling-edge interrupt to detect button presses, increments trigger counter in ISR, and main loop detects counter changes to toggle LED (GPIO44).

Comparison with Polling Approach:

Feature	External Interrupt (this chapter)	Polling (2. Basic Peripherals → 2. Button Detection)
Detection Mechanism	Hardware auto-triggers ISR	Software loop reads GPIO
CPU Efficiency	High (can sleep when idle)	Low (continuous polling)
Response Latency	Microsecond level	Depends on polling period (~20ms)
Coding Complexity	Need to understand ISR constraints	Simple and intuitive

Feature	External Interrupt (this chapter)	Polling (2. Basic Peripherals → 2. Button Detection)
Applicable Scenarios	Low power, real-time response	Simple applications, no power requirements

Typical Application Scenarios:

Application Type	Example
Edge Triggering	Emergency stop button, rotary speed sensor, PIR motion detection
Level Triggering	Power failure detection, overcurrent protection signal
ISR + Main Loop	Touch IC status pin, encoder Z-phase home signal

2. Core Concepts

2.1 Interrupt Trigger Types

ESP32-S3 GPIO supports 5 interrupt trigger types:

```
typedef enum {
    GPIO_INTR_DISABLE = 0, // Disable interrupt
    GPIO_INTR_POSEDGE = 1, // Rising edge trigger (0→1)
    GPIO_INTR_NEGEDGE = 2, // Falling edge trigger (1→0)
    GPIO_INTR_ANYEDGE = 3, // Any edge trigger (0→1 or 1→0)
    GPIO_INTR_LOW_LEVEL = 4, // Low level trigger
    GPIO_INTR_HIGH_LEVEL = 5, // High level trigger
} gpio_int_type_t;
```

Trigger Type	Trigger Condition	This Experiment
<code>NEGEDGE</code> (Falling)	Level transitions from 1→0	✅ Button press moment (pulled high → conducting low)
<code>POSEDGE</code> (Rising)	Level transitions from 0→1	Can be used for release detection
<code>ANYEDGE</code> (Any edge)	0→1 or 1→0	Need to determine direction by current level

Reason for selecting `GPIO_INTR_NEGEDGE` in this experiment: GPIO0 is configured with internal pull-up (default high), when button is pressed, level transitions from 1 to 0, forming a falling edge.

2.2 IRAM_ATTR — Why ISR Must Be in RAM

```
static void IRAM_ATTR exti_isr_cb(void *arg)
```

`IRAM_ATTR` places function code in ESP32-S3's **IRAM (internal SRAM)** instead of Flash. Reason:

Normal function call flow:
CPU → Cache → Flash (SPI) → return instruction

Risk when ISR triggers:
If Flash Cache miss and SPI bus is occupied
→ Need to wait for Flash read to complete
→ Interrupt response delay uncertain (possibly hundreds of μs)
→ Worst case: Cache miss + interrupt nesting → deadlock

IRAM_ATTR solution:
CPU → IRAM (SRAM) → immediately get instruction
→ Zero-wait response (deterministic delay)

Core Principle: ISR functions (and all sub-functions they call) must be marked with `IRAM_ATTR`, or linked to IRAM segment at compile time. ISR-related APIs in `driver/gpio.h` already have IRAM placement built-in.

2.3 volatile — Compiler Optimization Barrier

```
static volatile uint64_t intr_count = 0; // Modified by ISR, read by main loop
static volatile uint32_t led_state = 0; // Modified by ISR, read by main loop
```

Without volatile	With volatile
Compiler caches <code>intr_count</code> in register	Every access forces read from memory
<code>while(last_count != intr_count)</code> always reads old register value → may infinite loop	Each read gets latest value in memory
ISR modifications "invisible" to main loop	ISR modifications take effect immediately

`volatile` does not guarantee **atomicity**. For 32-bit variables on 32-bit processors, reads are atomic (single instruction), but 64-bit variable reads require two instructions. In this experiment, ISR only writes, main loop only reads, and no strict timing consistency is required, so no data race occurs.

2.4 ISR Design Principles

Things ISR can do:

- ✓ Increment counter
- ✓ Set flag
- ✓ Read/write volatile variables
- ✓ Call FreeRTOS API versions xxxFromISR()

Things ISR cannot do:

- ✗ printf() / ESP_LOGI() (blocking UART output)
- ✗ vTaskDelay() (ISR is not a task, cannot delay)
- ✗ Acquire mutex/semaphore (may deadlock)
- ✗ Heavy computation (extends interrupt response time)
- ✗ Operate Flash / NVS (SPI bus may be occupied)

3. Project Architecture and Flow

```
app_main()
├─ led_init()           ← Configure GPIO44 output, initially off
│
├─ exti_init()          ← Three-step interrupt configuration
│   ├─ gpio_config(.intr_type=NEGEDGE)  GPIO0 input+pull-up+falling edge
│   ├─ gpio_install_isr_service(0)      Install ISR unified dispatch service
│   └─ gpio_isr_handler_add()           Bind exti_isr_cb to GPIO0
│
└─ while(1) Main Loop
    ├─ vTaskDelay(100ms)
    └─ last_count != intr_count? → New interrupt
        ├─ gpio_set_level() → Update LED
        ├─ led_state = !led_state
        └─ last_count = intr_count
```

Interrupt trigger chain:

```
GPIO0 falling edge (button press)
→ GPIO hardware detection → generate interrupt signal
→ CPU save context → jump
    → gpio_isr_service() (IRAM)  Unified dispatch
    → exti_isr_cb() (IRAM)       User callback
    → intr_count++               Increment counter
    ← return to dispatcher
← CPU restore context
← continue executing interrupted code
```

4. Hardware Dependencies

Hardware Connection:

Pin	Peripheral	Configuration	Description
GPIO0	BOOT Button	Input + internal pull-up + falling edge interrupt	Pressed=low, interrupt triggers
GPIO44	LED	Push-pull output	Initially on, toggles after ISR triggers

Dependencies: `freertos`, `driver/gpio`, `esp_log`

5. Source Code Analysis

5.1 Macro Definitions and Global Variables

Use `#define` to give pins readable names, then define two `volatile` global variables for ISR and main loop to share data.

```
#define TAG                "EXTI_IRQ"           // Log tag
#define EXTI_TRIGGER_PIN   GPIO_NUM_0           // Interrupt pin: GPIO0
#define LED_PIN            GPIO_NUM_44          // LED pin: GPIO44

static volatile uint64_t intr_count = 0; // Interrupt trigger count (volatile: modified in ISR)
static volatile uint32_t led_state  = 0; // LED state (volatile: modified in ISR)
```

5.2 exti_isr_cb() — Interrupt Callback (Core)

This is the most core interrupt service function in the entire program—only one thing: increment counter by 1, nothing else.

```
static void IRAM_ATTR exti_isr_cb(void *arg)
{
    intr_count++; // Only increment count
}
```

This is the **innermost function** of the entire interrupt system, doing only one thing: increment counter. Why so simple?

1. **ISR should be as short as possible** — 1 increment instruction < 1µs
2. **All complex logic handed to main loop** — Decouple interrupt response from business processing
3. **Usage of `arg` parameter** — The 3rd parameter of `gpio_isr_handler_add()` can pass a context pointer (NULL in this experiment), used to distinguish trigger sources when one ISR corresponds to multiple GPIOs

5.3 led_init() — LED Initialization

Fill a GPIO configuration struct, set LED pin to push-pull output, pull-up/down disabled, interrupt disabled, and after completion default to low to keep it on.

```
static void led_init(void)
{
    gpio_config_t led_conf = {
        .pin_bit_mask = (1ULL << LED_PIN),
        .mode          = GPIO_MODE_OUTPUT,
        .pull_up_en     = GPIO_PULLUP_DISABLE,
        .pull_down_en   = GPIO_PULLDOWN_DISABLE,
        .intr_type      = GPIO_INTR_DISABLE,
    };
    gpio_config(&led_conf);
    gpio_set_level(LED_PIN, 0);    // Initially on
    led_state = 0;
}
```

5.4 exti_init() — Interrupt Initialization (Three Steps)

Interrupt initialization in three steps: first configure GPIO0 as input+pull-up+falling edge trigger, then install a global ISR dispatcher, finally attach our callback function to GPIO0.

```
static void exti_init(void)
{
    // === Step 1: Configure GPIO0 as interrupt input pin ===
    gpio_config_t exti_conf = {
        .pin_bit_mask = (1ULL << EXTI_TRIGGER_PIN),
        .mode          = GPIO_MODE_INPUT,
        // Internal pull-up, default HIGH
        .pull_up_en     = GPIO_PULLUP_ENABLE,
        .pull_down_en   = GPIO_PULLDOWN_DISABLE,
        // Falling edge trigger
        .intr_type      = GPIO_INTR_NEGEDGE,
    };
    gpio_config(&exti_conf);

    // === Step 2: Install GPIO ISR unified dispatch service ===
    gpio_install_isr_service(0);    // flags=0: default priority

    // === Step 3: Bind our callback to GPIO0 ===
    gpio_isr_handler_add(EXTI_TRIGGER_PIN, exti_isr_cb, NULL);
}
```

`gpio_install_isr_service(0)` **should only be called once**. It creates a unified ISR dispatcher, all GPIO interrupts are dispatched through this dispatcher to their respective callbacks. Multiple calls return `ESP_ERR_INVALID_STATE`. This experiment's `exti_init()` is also called only once, executed during initialization phase from `app_main()`.

Meaning of flags parameter: 0 = use default priority. Optional `ESP_INTR_FLAG_IRAM` (ISR itself in IRAM), `ESP_INTR_FLAG_LEVEL1/2/3`, etc., for higher priority interrupt requirements.

5.5 app_main() — Main Entry

The main function called automatically by FreeRTOS after boot, does three things: initialize LED, initialize external interrupt, then in an infinite loop check interrupt count every 100ms, toggle LED if changed.

```
void app_main(void)
{
    ESP_LOGI(TAG, "ESP32-S3 External Interrupt LED Control Demo Start");
    led_init();
    ESP_LOGI(TAG, "LED initialized on GPIO%d", (int)LED_PIN);
    exti_init();
    ESP_LOGI(TAG, "EXTI initialized on GPIO0 (falling edge), waiting for trigger...");

    int last_count = 0;
    while (1) {
        vTaskDelay(100 / portTICK_PERIOD_MS);

        // New interrupt detected
        if (last_count != intr_count) {
            gpio_set_level(LED_PIN, led_state);
            vTaskDelay(100 / portTICK_PERIOD_MS);
            led_state = !led_state;
            last_count = intr_count;
        }
    }
}
```

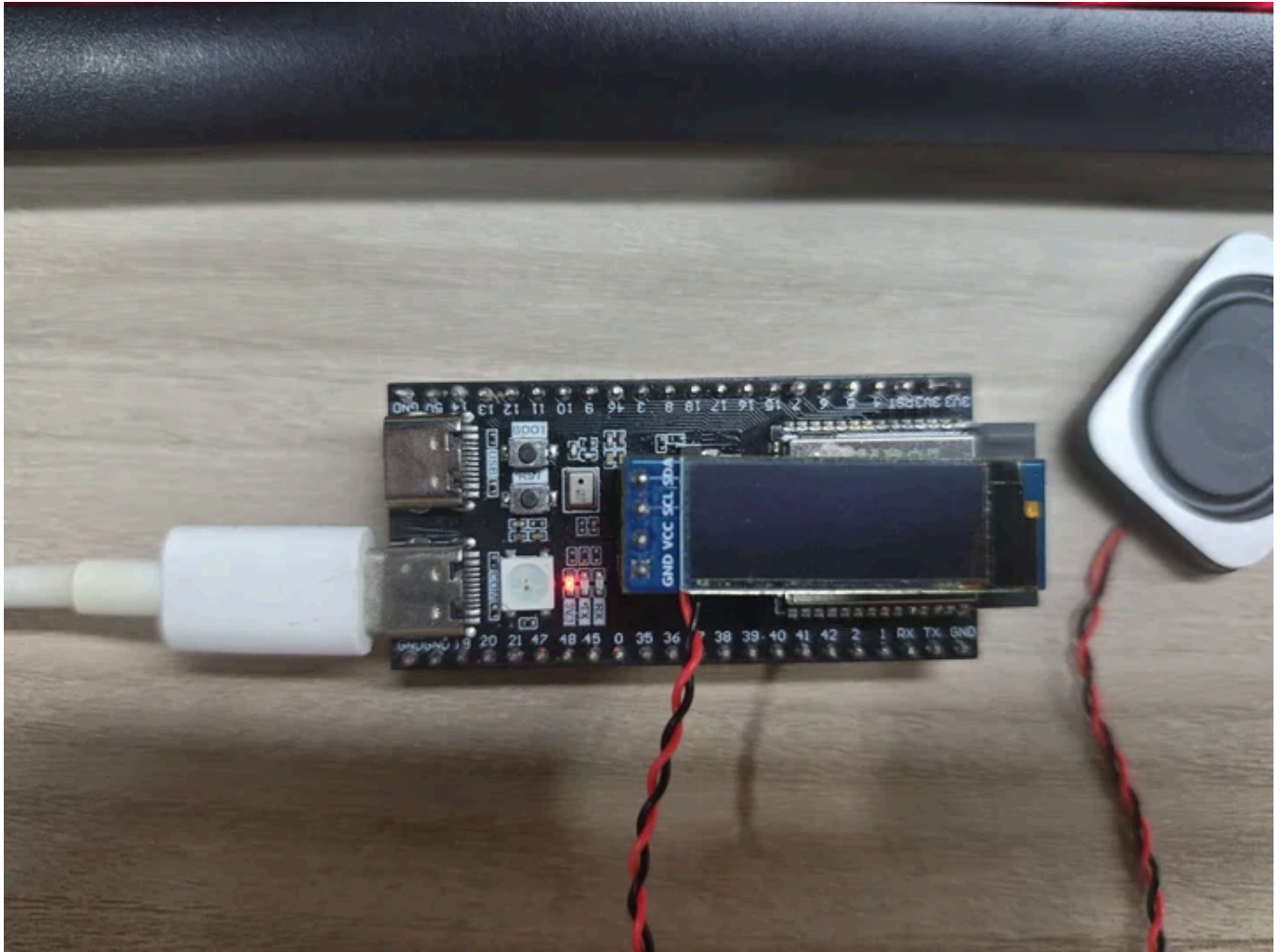
Why `gpio_set_level()` + `vTaskDelay()` before toggling LED? These two statements implement simple "ON for 100ms" effect, ensuring LED state change is visible to the naked eye. If button is pressed too fast (interrupt frequency > 10Hz), LED may not keep up with every toggle—but `intr_count` still correctly records all triggers.

6. Operation Flow

6.1 Build and Flash

For detailed flashing process, see: 1. Before Development → 3. Build and Flash

6.2 Normal Operation



6.3 Verification Testing

Test Item	Operation	Expected
Basic function	Press BOOT button	LED toggles
Rapid continuous press	Continuous fast clicks	Each press triggers (ISR doesn't miss interrupts)
Long press	Hold down	ISR triggers only once on falling edge (LED toggles once)
Power-on default	Don't press button	LED on

7. Common Issues

Phenomenon	Cause	Solution
ISR not triggered	<code>intr_type</code> misconfigured / <code>gpio_isr_handler_add</code> not called	Confirm <code>GPIO_INTR_NEGEDGE</code> + callback registered
Button triggers multiple interrupts	Mechanical bounce (falling edge + bouncing creates multiple edges)	Debounce outside ISR, or add time window filter in ISR

Phenomenon	Cause	Solution
Program crash (Guru Meditation Error)	Called non-ISR-safe API in ISR	Remove printf, vTaskDelay, etc. from ISR
<code>gpio_install_isr_service</code> fails	ISR service already installed	Ensure installed only once globally (or use <code>ESP_ERROR_CHECK</code>)
LED flashing unstable	Main loop 100ms polling + ISR timing competition	Use <code>vTaskNotifyGiveFromISR()</code> instead of polling
Large interrupt response delay	Flash Cache miss + function in Flash	Add <code>IRAM_ATTR</code> to ISR function, use <code>ESP_INTR_FLAG_IRAM</code>